

AWS · EKS · FINOPS

EKS Cost Optimization

Cut your Kubernetes bill by
40-70% without breaking production



10 battle-tested strategies

Spot Instances

Karpenter

Right-Sizing

KEDA

Savings Plans

Networking

Fargate

Cost Visibility

Introduction — Why EKS Bills Spiral Out of Control

Amazon EKS is one of the most powerful managed Kubernetes services available, but without deliberate configuration and continuous tuning, engineering teams routinely overpay by 40–60% or more every single month. The root causes are almost always the same handful of issues: over-provisioned nodes, idle workloads running 24/7, wrong instance families, poor autoscaling strategies, and hidden data transfer fees that nobody notices until the bill arrives.

This guide walks you through every major cost lever available in EKS — from quick wins that take under 30 minutes to architectural changes that compound savings over months. Each section includes the tool, the configuration, real numbers, and the reasoning behind the recommendation.

💡 Who this guide is for

DevOps engineers, SREs, and platform engineers running production EKS workloads. Familiarity with Kubernetes basics (Deployments, Services, node groups) is assumed. AWS IAM and VPC knowledge is helpful but not required for most sections.

The Five Root Causes of EKS Overspending

Root Cause	Typical Waste	Primary Fix
Over-provisioned pod resources	20–40% of compute	VPA + Goldilocks right-sizing
Running On-Demand instead of Spot	30–90% per node	Spot instances + Karpenter
No scale-to-zero for idle workloads	10–70% per service	KEDA event-driven autoscaling
Unused On-Demand baseline (no commitment)	30–60% of baseline	Compute Savings Plans
Hidden data transfer & networking fees	5–25% of total bill	VPC Endpoints + topology awareness

Strategy 1: Spot Instances — Your Biggest Lever

Spot Instances are spare EC2 capacity available at up to 90% off On-Demand pricing. AWS can reclaim them with a 2-minute warning, which means they require workload design for interruption tolerance — but for the right workloads, the savings are transformational.

🔗 Expected savings: 30–90% on eligible node groups

A single m5.xlarge On-Demand instance costs roughly \$140/month. The same instance as Spot typically runs \$14–40/month depending on availability zone and pool pressure.

1.1 Classifying Your Workloads

The first step is honest classification. Not every workload is Spot-safe, and running Spot on the wrong things causes outages. Use this framework:

✅ Spot-Safe Workloads

- CI/CD runners and GitHub Actions build agents
- Batch processing, data pipelines, ETL jobs
- Stateless microservices (with proper PDBs)
- ML training and inference workloads (with retry logic)
- Dev, staging, and QA environments (100% Spot is fine)
- Queue consumers that can restart mid-job
- Scheduled jobs and cron-based workloads

❌ On-Demand / Reserved Only

- Databases running on EC2 (use RDS instead)
- Kafka brokers and ZooKeeper nodes
- Single-replica critical services (no PDB configured)
- StatefulSets with persistent volumes requiring stable node assignment
- kube-system daemonsets and cluster add-ons
- Services with sub-second failover requirements

1.2 Instance Diversification Strategy

The most common Spot mistake is using only one instance family. When AWS reclaims capacity, it does so pool-by-pool, meaning all your m5.xlarge Spot nodes might vanish simultaneously. The solution is to spread across multiple families and sizes with similar performance characteristics:

Instance Family	vCPU	RAM	Relative Price	Best For
m5.xlarge	4	16 GB	Baseline	General workloads
m5a.xlarge	4	16 GB	-10%	AMD, same workload
m5n.xlarge	4	16 GB	+5%	Network-intensive
m6i.xlarge	4	16 GB	~Baseline	Latest Intel gen
m6a.xlarge	4	16 GB	-10%	Latest AMD gen
m4.xlarge	4	16 GB	-5%	Older gen fallback

**Rule: Use at least 3 instance families per Spot node group**

More Spot pools = more capacity = fewer interruptions. When one pool is reclaimed, Karpenter or the Cluster Autoscaler draws from the remaining pools automatically. Aim for 5+ families on critical Spot node groups.

1.3 PodDisruptionBudgets — Non-Negotiable

Every Deployment running on Spot nodes must have a PodDisruptionBudget (PDB). Without one, a Spot interruption can take down all replicas of a service simultaneously. A PDB tells Kubernetes how many pods can be unavailable at once during voluntary disruptions (which Spot interruption drains qualify as):

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: checkout-api-pdb
  namespace: production
spec:
  minAvailable: 2          # Always keep at least 2 pods running
  # OR use maxUnavailable instead:
  # maxUnavailable: 1      # Allow at most 1 pod to be unavailable
  selector:
    matchLabels:
      app: checkout-api
```

⚠ Target minimum 70% availability in your PDB

If you have 3 replicas, set minAvailable: 2. If you have 10 replicas, setting maxUnavailable: 3 gives you 70% minimum availability. Going below 50% defeats the purpose of the PDB entirely.

1.4 Mixed Node Group Architecture

The recommended production pattern is a three-tier node group architecture that ensures critical infrastructure stays stable while maximizing Spot usage for everything else:

Node Group	Instance Type	Capacity	Purpose	Workloads
system-ng	m5.large	On-Demand	Cluster control plane add-ons	kube-system, CoreDNS, CNI, CSI
app-spot-ng	m5/m5a/m6i.xl	Spot	Stateless application pods	All microservices with PDBs
batch-spot-ng	c5/c5a/c6i.2xl	Spot	CPU-intensive batch jobs	CI/CD, data processing, ML
critical-ng	m5.2xlarge	On-Demand	Stateful and critical svcs	Kafka, Redis, critical APIs

Strategy 2: Karpenter — Next-Generation Autoscaling

Karpenter is an open-source Kubernetes cluster autoscaler built by AWS that replaces the traditional Cluster Autoscaler + Managed Node Groups model. Instead of scaling pre-configured node groups, Karpenter observes pending pods and provisions exactly the right EC2 instance for each workload — often within 30 seconds.



Expected savings: 20–40% reduction in compute waste

Karpenter's bin-packing algorithm places pods much more efficiently than the Cluster Autoscaler. Its consolidation feature continuously monitors for underutilized nodes and replaces them with smaller, cheaper ones — even while your cluster is running.

2.1 Karpenter vs Cluster Autoscaler — Full Comparison

Feature	Cluster Autoscaler	Karpenter
Node provisioning speed	2–5 minutes	~30 seconds
Instance selection	Fixed node group	Any EC2 instance type
Spot instance diversification	Manual config per ASG	Automatic across all pools
Node bin-packing	Basic	Aggressive, real-time
Node consolidation	Limited	Continuous, automatic
Binds to specific AZ	Yes (per ASG)	No — selects optimal AZ
Custom scheduling constraints	Limited	Full Kubernetes constraints
GPU / ARM support	Manual node group	Automatic per pod request
Configuration complexity	High (many ASGs)	Low (NodePool + EC2NodeClass)

2.2 Core Karpenter Concepts

NodePool

A NodePool defines what Karpenter is allowed to provision. It specifies instance families, capacity types (On-Demand or Spot), availability zones, architecture (amd64/arm64), and consolidation behavior. Think of it as the policy that governs node provisioning.

```
apiVersion: karpenter.sh/v1
kind: NodePool
metadata:
  name: app-nodepool
spec:
  template:
    spec:
      requirements:
        - key: karpenter.sh/capacity-type
```

```
operator: In
values: ["spot", "on-demand"] # Prefer Spot, fall back to OD
- key: kubernetes.io/arch
operator: In
values: ["amd64"]
- key: node.kubernetes.io/instance-type
operator: In
values:
["m5.xlarge", "m5a.xlarge", "m6i.xlarge", "m6a.xlarge", "m5.2xlarge"]
nodeClassRef:
group: karpenter.k8s.aws
kind: EC2NodeClass
name: app-nodeclass
disruption:
consolidationPolicy: WhenUnderutilized
consolidateAfter: 30s
expireAfter: 720h # Rotate nodes every 30 days
limits:
cpu: 1000 # Max 1000 vCPUs in this pool
memory: 4000Gi
```

EC2NodeClass

EC2NodeClass maps to the underlying AWS infrastructure: the AMI, launch template, subnet selection, security groups, and IAM instance profile. One NodeClass can serve multiple NodePools.

```
apiVersion: karpenter.k8s.aws/v1
kind: EC2NodeClass
metadata:
name: app-nodeclass
spec:
amiFamily: AL2023 # Amazon Linux 2023
role: KarpenterNodeRole # IAM role for EC2 instances
subnetSelectorTerms:
- tags:
karpenter.sh/discovery: my-cluster
securityGroupSelectorTerms:
- tags:
karpenter.sh/discovery: my-cluster
tags:
Team: platform
Environment: production
```

2.3 Consolidation — The Hidden Superpower

Consolidation is Karpenter's most impactful feature. When enabled, Karpenter continuously evaluates your cluster and asks: 'Can I move all pods from this underutilized node to other existing nodes, and terminate the empty one?' If yes, it does so automatically — even during active traffic, subject to PodDisruptionBudgets.

The consolidation policies available are:

- **WhenUnderutilized** — Karpenter actively replaces underutilized nodes with smaller ones, or consolidates pods and removes empty nodes. This is the most aggressive and most cost-effective setting.
- **WhenEmpty** — Karpenter only removes nodes that have zero pods running on them. Safer, but misses the right-sizing opportunity.

💡 Enable consolidation on all non-critical node pools

Set `consolidationPolicy`: `WhenUnderutilized` and `consolidateAfter`: 30s. In a typical cluster, this alone reduces node count by 15–25% within the first 24 hours as natural load variation creates consolidation opportunities.

2.4 Spot Interruption Handling

Karpenter natively handles Spot interruptions through AWS EC2 Instance Action notifications. When AWS signals an interruption, Karpenter immediately begins cordoning and draining the affected node, ensuring workloads are rescheduled before the 2-minute window expires. No external tooling (like node-termination-handler) is needed when using Karpenter.

Strategy 3: Right-Sizing Pods and Nodes

Resource right-sizing is the practice of matching CPU and memory requests/limits to what workloads actually consume — not what developers guessed when they first wrote the Deployment manifest. Studies of production clusters consistently show that teams set requests 2–3× higher than actual P95 usage, leading to 40–60% of requested cluster capacity being permanently idle.

🔗 Expected savings: 15–30% reduction in compute costs

Right-sizing works by reducing the effective resource requests pods make, which allows the autoscaler to schedule more pods per node and provision fewer (or smaller) nodes overall. Combined with Karpenter, the effect compounds.

3.1 Understanding Resource Requests vs Limits

Kubernetes uses two resource specifications for CPU and memory:

Spec	Effect on Scheduling	Effect on Runtime	Getting it Wrong
requests	Used by scheduler to place pod on node	Guaranteed minimum allocation	Too high = wasted capacity, too low = noisy neighbor
limits	Not used for scheduling	Hard cap — pod is throttled (CPU) or OOMKilled (memory)	Too low = OOMKill loops, too high = no protection

⚠️ Never leave memory limits unset in production

Without a memory limit, a pod can consume all available node memory, triggering OOMKills across the entire node. Every production pod must have a memory limit set to at least P99 actual usage + 20% headroom.

3.2 Measurement — The 7-Day Baseline

Right-sizing requires real usage data. Collect at least 7 days of metrics (ideally 30) that capture typical business cycles including weekday peaks, weekend troughs, and any batch processing windows. The key percentiles to measure are:

- **CPU requests:** Set to P95 of actual CPU usage over the measurement window. This ensures 95% of the time the pod has enough CPU without throttling.
- **Memory requests:** Set to P90 of actual memory usage. Memory typically grows over time (due to caching), so lean slightly higher than CPU.
- **CPU limits:** Set to 2–3× the request. CPU throttling is preferable to OOMKill, so give some headroom.
- **Memory limits:** Set to P99 actual usage + 20% buffer. This is your safety net against memory leaks.

3.3 Tools for Right-Sizing

VPA (Vertical Pod Autoscaler)

VPA analyzes historical resource usage and provides recommendations for right-sizing. It can operate in three modes:

- **Off mode:** Only computes recommendations, applies nothing. Best starting point — review before committing.
- **Initial mode:** Applies recommendations only when pods are first scheduled. No live disruption.
- **Auto mode:** Continuously applies recommendations, which requires pod restarts. Use with caution in production.

```
apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
metadata:
  name: checkout-api-vpa
  namespace: production
spec:
  targetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: checkout-api
  updatePolicy:
    updateMode: "Off"    # Start with Off, review recommendations
  resourcePolicy:
    containerPolicies:
      - containerName: checkout-api
        minAllowed:
          cpu: 50m
          memory: 64Mi
        maxAllowed:
          cpu: 2
          memory: 2Gi
```

Goldilocks by Fairwinds

Goldilocks is a free open-source tool that runs VPA in recommendation mode across all namespaces and presents the results in a clean web dashboard. It shows current requests vs VPA-recommended requests for every container in the cluster, making it the fastest way to identify over-provisioned workloads without making any changes.

```
# Install Goldilocks
helm install goldilocks fairwinds-stable/goldilocks \
  --namespace goldilocks --create-namespace

# Enable namespace
kubectl label namespace production goldilocks.fairwinds.com/enabled=true

# Access dashboard
kubectl port-forward svc/goldilocks-dashboard -n goldilocks 8080:80
```

Strategy 4: KEDA — Scale to Zero

KEDA (Kubernetes Event-Driven Autoscaler) extends Kubernetes' built-in HPA to support scaling based on external event sources — and critically, scaling all the way down to zero replicas when there is no work to do. Zero pods means zero resource requests, which means Karpenter terminates empty nodes. The combined effect is that entire workloads and their infrastructure cost nothing during off-hours.

💧 Expected savings: 40–70% for eligible workloads

A t3.medium queue consumer running 24/7 costs ~\$25/month. The same workload with KEDA scaling to zero during 16 off-hours costs ~\$8/month — a 68% reduction per service. Multiply across 10–20 services and savings become significant.

4.1 Why HPA Cannot Do This

The built-in Horizontal Pod Autoscaler has a hard minimum of 1 replica by design — it cannot scale below that. This means even an idle service with zero traffic still consumes a node slot 24/7. KEDA lifts this restriction by integrating directly with event sources and allowing `minReplicaCount: 0`.

Capability	HPA	KEDA
Minimum replicas	1 (hard floor)	0 (true scale-to-zero)
Scale trigger sources	CPU / Memory only	SQS, Kafka, HTTP, Cron, Prometheus, 60+ scalers
Scale based on queue depth	No	Yes — native SQS/Kafka scalers
Cron scheduling	No	Yes — built-in Cron scaler
Cold-start latency	N/A (always 1+ replica)	Configurable — typically 5–30s

4.2 KEDA Scalars for Common AWS Patterns

SQS Queue Consumer

The most common KEDA pattern: scale consumers based on the number of messages in an SQS queue. When the queue is empty, consumers scale to zero. When messages arrive, KEDA scales up proportionally.

```
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata:
  name: sqs-consumer-scaler
  namespace: production
spec:
  scaleTargetRef:
    name: order-processor
  minReplicaCount: 0      # Scale all the way to zero
  maxReplicaCount: 50    # Max burst capacity
  cooldownPeriod: 300    # Wait 5 min before scaling to zero
```

```
triggers:
  - type: aws-sqs-queue
    authenticationRef:
      name: keda-aws-credentials
    metadata:
      queueURL: https://sqs.ap-south-1.amazonaws.com/123456/orders
      queueLength: "5" # Target 5 messages per replica
      awsRegion: ap-south-1
```

Cron Scaler — Guaranteed Zero Overnight

For workloads with predictable traffic patterns, the Cron scaler provides a hard schedule: scale up before peak, scale to zero after. This is the safest KEDA pattern since it doesn't depend on external signals.

```
triggers:
  - type: cron
    metadata:
      timezone: Asia/Kolkata
      start: 30 8 * * 1-5 # Scale up at 8:30 AM weekdays
      end: 0 20 * * 1-5 # Scale to zero at 8:00 PM weekdays
      desiredReplicas: "10"
```

4.3 KEDA + Karpenter — The Full Stack

The maximum cost reduction comes from combining KEDA with Karpenter. The sequence works as follows:

1. Queue empties → KEDA detects zero messages → KEDA scales deployment to 0 replicas
2. Pods are terminated → nodes become empty
3. Karpenter detects empty nodes → terminates them after `consolidateAfter` delay
4. EC2 instances are returned to AWS → billing stops within minutes
5. New message arrives → KEDA scales up → Karpenter provisions new node in ~30s → pods scheduled

⚡ Cold-start latency budget

The typical KEDA + Karpenter cold-start (0 → serving traffic) takes 45–90 seconds: 30s for Karpenter to provision a node, 15–30s for pod startup and readiness. For SLO-sensitive workloads, keep `minReplicaCount`: 1 during business hours using a Cron scaler and allow scale-to-zero only overnight.

Strategy 5: Savings Plans and Reserved Instances

For your baseline On-Demand workloads — the ones that must run 24/7 regardless of traffic — committing to a Savings Plan is the simplest and most reliable cost reduction available. Unlike Spot, there's no interruption risk and no architectural change required. You simply commit to spending a fixed amount per hour in exchange for a substantial discount.

Expected savings: 30–72% on committed On-Demand spend

A Compute Savings Plan at a 1-year no-upfront commitment typically delivers 40–50% savings versus On-Demand. A 3-year all-upfront commitment can reach 60–66%. EC2 Instance Savings Plans (less flexible, but deeper discount) can reach 72%.

5.1 Savings Plan Types Compared

Plan Type	Max Discount	Flexibility	Covers Fargate	Best For
Compute Savings Plan (1yr)	~66%	Any EC2, Fargate, Lambda, any region	Yes	Mixed/evolving workloads
EC2 Instance Savings Plan (1yr)	~72%	Fixed family + region, any size/OS	No	Stable node family
Reserved Instance (1yr)	~40%	Fixed type, AZ, OS	No	Very stable DB nodes
Reserved Instance (3yr)	~60%	Fixed type, AZ, OS	No	Long-term baseline

5.2 Sizing Your Commitment

The most common mistake is over-committing. If you commit to \$50/hour and your On-Demand usage drops (due to Spot adoption, KEDA scale-to-zero, or load reduction), the commitment still applies and you're effectively paying for unused capacity. The safe approach:

- **Step 1:** Review your last 3 months of EC2 On-Demand spend in AWS Cost Explorer. Find the minimum monthly spend — this is your safe baseline.
- **Step 2:** Commit to 70–80% of that minimum. The remaining 20–30% is your headroom buffer for unexpected On-Demand usage.
- **Step 3:** Choose 1-year no-upfront if you're still optimizing. Switch to 3-year all-upfront once your baseline is stable.
- **Step 4:** Use the AWS Cost Explorer Savings Plans recommendations tool — it does the math for you based on actual usage.

Use AWS Cost Explorer Savings Plans Recommendations

Navigate to AWS Cost Explorer → Savings Plans → Recommendations. Set lookback period to 30 days and coverage threshold to 80%. AWS will calculate the exact hourly commitment that maximizes savings without over-committing. Start with the recommended amount and adjust after 3 months.

Strategy 6: Networking Cost Reduction

Data transfer charges are the most frequently overlooked cost driver in EKS. They don't appear as a large single line item — they accumulate across dozens of small flows that individually seem insignificant but collectively can represent 10–25% of your total AWS bill. The four main drains are cross-AZ traffic, NAT Gateway egress, per-service load balancers, and CloudWatch Logs ingestion.

6.1 Cross-AZ Traffic — \$0.01/GB Each Way

AWS charges \$0.01/GB for data transferred between availability zones in the same region. In a busy microservices cluster, service-to-service calls crossing AZ boundaries add up quickly. A single service making 10 GB/day of cross-AZ calls costs \$73/month — multiply that by 20 services and you're at \$1,460/month in avoidable charges.

Fix: Topology-Aware Scheduling

Use topologySpreadConstraints to keep pods in the same AZ as their callers, and configure your Services to prefer same-AZ endpoints with topology hints:

```
# Deployment — spread pods evenly across AZs
spec:
  template:
    spec:
      topologySpreadConstraints:
        - maxSkew: 1
          topologyKey: topology.kubernetes.io/zone
          whenUnsatisfiable: DoNotSchedule
          labelSelector:
            matchLabels:
              app: checkout-api

# Service — enable topology-aware routing (K8s 1.27+)
spec:
  trafficDistribution: PreferClose
```

6.2 NAT Gateway Egress — \$0.045/GB

Every AWS API call from your pods (to S3, ECR, SSM, CloudWatch, STS, etc.) that goes through a NAT Gateway costs \$0.045/GB. A cluster pulling Docker images from ECR several times per hour across 50 nodes can generate hundreds of GB per month in NAT charges — all of which can be eliminated with VPC Endpoints.

Fix: VPC Endpoints

VPC Endpoints allow traffic to AWS services to stay within the AWS network, bypassing NAT Gateway entirely. The S3 and DynamoDB Gateway Endpoints are free. Interface Endpoints cost ~\$7/month per AZ but pay for themselves immediately on any moderately busy cluster.

VPC Endpoint	Type	Monthly Cost	Primary Traffic Eliminated
S3	Gateway	Free	Image layers from ECR (stored in S3), log uploads, artifact storage
ECR API	Interface	~\$7/AZ/month	Docker pull authentication, image manifest fetches
ECR DKR	Interface	~\$7/AZ/month	Docker image layer downloads
SSM	Interface	~\$7/AZ/month	Parameter store reads, session manager connections
CloudWatch Logs	Interface	~\$7/AZ/month	All pod log shipping via Fluent Bit or CloudWatch agent
STS	Interface	~\$7/AZ/month	AWS credential refreshes (IRSA token exchanges)
EKS	Interface	~\$7/AZ/month	Control plane API calls from nodes

6.3 Per-Service Load Balancers — \$18+/Month Each

Every Kubernetes Service of type LoadBalancer creates a Classic or Network Load Balancer. At \$16–18/month per load balancer plus data processing charges, a cluster with 30 services can rack up \$500+/month in load balancer costs alone. The solution is to consolidate all external-facing services behind a single Application Load Balancer using the AWS Load Balancer Controller.

🌐 Single ALB = one \$16/month charge for all services

The AWS Load Balancer Controller enables Ingress objects that create one ALB shared across all services, using path-based or host-based routing to direct traffic to the correct backend. 30 services with individual LBs = ~\$540/month. 30 services behind one ALB Ingress = ~\$20/month.

6.4 CloudWatch Logs — \$0.50/GB Ingested

CloudWatch charges \$0.50/GB for log ingestion. A busy cluster generating 50 GB/day of container logs costs \$750/month in ingestion alone, plus storage. The alternative is to ship logs directly to S3 using Fluent Bit (free, runs as a DaemonSet) and query them with Athena on demand:

- **Cost to store 1 TB in S3:** ~\$23/month
- **Cost to store 1 TB in CloudWatch:** ~\$500/month in ingestion + \$300/month in storage = \$800/month
- **Athena query cost:** \$5/TB scanned — typically under \$10/month for ops usage
- **Net saving:** ~\$770/month per TB of logs

Strategy 7: Fargate Positioning and Dev Cluster Shutdown

7.1 When to Use Fargate vs EC2

AWS Fargate for EKS eliminates node management entirely — pods get their own isolated compute, no nodes to patch, no node groups to configure. The trade-off is cost: Fargate typically costs 2–3× more per vCPU/GB than equivalent EC2 Spot. Use it selectively:

Use Fargate when:

- Short-lived batch jobs with variable resource needs
- Security-sensitive workloads needing pod-level isolation
- Dev/test where node patching overhead isn't worth it
- Bursty event workloads with unpredictable scaling needs
- Teams without expertise to manage EC2 node groups

Use EC2 Spot instead when:

- High-traffic stateless services (3–5× cheaper than Fargate)
- GPU or accelerated compute workloads
- Workloads requiring DaemonSets or hostPath volumes
- Long-running pods with stable resource patterns
- Anything where cost is the primary driver

7.2 Dev Cluster Auto-Shutdown — Up to 80% Savings

Development and staging clusters are the single easiest source of savings in most organizations. A dev cluster that runs 24/7 but is only used 8 hours per day is paying for 16 wasted hours — 67% of its compute bill. Three proven approaches:

Strategy	Tool	Est. Savings	Complexity	Best For
Schedule EC2 stop/start	AWS Instance Scheduler	60–70%	Low	Fixed-schedule teams
Scale deployments to zero	kube-downscaler	65–75%	Low	Teams needing quick cluster access
Ephemeral clusters per PR	Terraform + CI/CD	75–85%	High	Platform teams with GitOps

kube-downscaler Configuration

```
# Install kube-downscaler
helm install kube-downscaler kube-downscaler/kube-downscaler \
  --namespace kube-system \
  --set schedule="Mon-Fri 08:00-20:00 Asia/Kolkata"

# Annotate namespaces to exclude from downscaling
```

```
kubectl annotate namespace kube-system downscaler/exclude=true

# Opt a specific deployment out of downscaling
kubectl annotate deployment critical-app downscaler/exclude=true
```


Strategy 8: Cost Visibility and Attribution

You cannot sustainably optimize what you cannot measure. Cost visibility means knowing exactly which team, namespace, workload, and service is responsible for every dollar of EKS spend — and having that data update frequently enough to act on it before the next bill arrives.

8.1 Tagging Strategy — The Foundation

AWS cost allocation tags are the backbone of any attribution system. Every resource that generates cost must be tagged consistently. Define your tag schema once and enforce it via AWS Config rules and tag policies:

Tag Key	Example Values	Purpose
Team	platform-eng, payments, checkout	Which engineering team owns this
Environment	production, staging, dev	Which environment tier
Project	checkout-api, data-pipeline	Which product or project
CostCenter	cc-1042, cc-0815	Finance cost center mapping
ManagedBy	terraform, karpenter, manual	How the resource was provisioned

⚠️ Tags must be applied at the node group level AND as Kubernetes labels

AWS billing sees EC2 tags. Kubernetes cost tools (Kubecost, OpenCost) see pod labels. For attribution to work end-to-end, mirror your AWS tag schema as Kubernetes labels on all Deployments and Namespaces.

8.2 Cost Monitoring Tools

Tool	Cost	Key Strength	Best For
Kubecost	Free (limited) / \$499+/mo	Real-time per-workload Kubernetes cost breakdown	Teams wanting deep K8s cost visibility
OpenCost	Free (CNCF)	Open-source, label-based cost allocation	Teams avoiding vendor lock-in
AWS Cost Explorer	Free	Native AWS cost grouping, budget alerts	High-level account-wide analysis
CloudWatch Container Insights	Paid per metric	Node + pod metrics correlated with cost	Combining cost with

Tool	Cost	Key Strength	Best For
			performance data

8.3 Monthly Cost Review Ritual

Optimization without review is unsustainable. Establish a monthly 30-minute cost review with this structure:

- 6. Open Kubecost — sort namespaces by monthly cost, descending. Identify the top 5 cost drivers.
- 7. For each top namespace: compare requested vs actual CPU/memory. Flag anything with >50% waste.
- 8. Check Karpenter's consolidation metrics — how many nodes were consolidated last month?
- 9. Review Spot interruption rate — if >5%, diversify instance families.
- 10. Check Savings Plan utilization — is coverage above 70%? Are there new On-Demand workloads to commit?
- 11. Create one right-sizing ticket per top cost driver. Track savings week-over-week.

The Complete Action Checklist

Use this table to track your EKS cost optimization progress. Work through them in order — items at the top deliver the highest ROI for the effort required.

#	Action	Est. Savings	Effort	Status
1	Move Spot-safe workloads to Spot instances	30–90%	Medium	☐
2	Install Karpenter + enable WhenUnderutilized consolidation	20–40%	Medium	☐
3	Right-size pods with VPA + Goldilocks (7-day measurement)	15–30%	Low	☐
4	Add KEDA for queue consumers and batch jobs (scale to 0)	40–70%	Medium	☐
5	Purchase Compute Savings Plan for On-Demand baseline	30–66%	Low	☐
6	Add VPC Gateway Endpoint for S3 (free, immediate)	5–15%	Low	☐
7	Add Interface Endpoints for ECR, SSM, CloudWatch, STS	5–15%	Low	☐
8	Consolidate all services behind single ALB Ingress	5–15%	Medium	☐
9	Enable topology-aware routing to reduce cross-AZ traffic	3–10%	Low	☐
10	Ship logs to S3 via Fluent Bit instead of CloudWatch Logs	10–20%	Medium	☐
11	Schedule dev cluster auto-shutdown via kube-downscaler	60–80%	Low	☐
12	Tag all node groups (Team/Environment/Project/CostCenter)	Visibility	Low	☐
13	Deploy Kubecost or OpenCost for per-namespace attribution	Visibility	Low	☐
14	Establish monthly 30-min cost review ritual	Ongoing	Low	☐

Common Mistakes and How to Avoid Them

Mistake	Impact	The Fix
Using Spot for stateful workloads without PVCs that survive node loss	Data loss, service outages	Move stateful workloads to On-Demand node groups; use EBS Multi-Attach or EFS
Setting memory limits too low, causing constant OOMKills	Cascading restarts, degraded SLOs	Set limits to P99 + 20% buffer; use VPA Auto mode carefully
Over-committing Savings Plans based on current (pre-optimization) spend	Paying for unused commitments	Commit to 70–80% of post-optimization minimum baseline
Enabling Karpenter consolidation without PodDisruptionBudgets	Unexpected pod evictions in production	Add PDBs to all production Deployments before enabling consolidation
Leaving CloudWatch Log retention set to Never Expire	Unbounded storage costs compounding monthly	Set retention to 30–90 days; archive older logs to S3
Ignoring cross-AZ traffic until the bill arrives	Hidden 10–20% networking surcharge	Enable topology hints and topologySpreadConstraints proactively
Not tagging resources from day one	Unable to attribute costs, impossible to optimize fairly	Enforce tags via AWS Config + tag policies before first workload deploys

Final Thoughts

EKS cost optimization is not a one-time project — it is a continuous discipline. The strategies in this guide compound over time: right-sizing pods allows Karpenter to fit more on fewer nodes, which allows Savings Plans to cover a smaller baseline, which reduces the exposure to On-Demand pricing, which makes Spot the only logical choice for the remainder.

Start with Spot Instances and Karpenter. Those two changes alone will cut most clusters' bills in half within a week. Then layer in KEDA for idle workloads, right-sizing for efficiency, and Savings Plans for the committed baseline. Add cost visibility from day one so you can measure every change.

The teams that treat cost as a first-class engineering concern — alongside reliability, security, and performance — consistently outperform those that treat it as a finance problem to solve at billing time.



Follow @devopsshack for hands-on DevOps content

YouTube tutorials, Instagram carousels, LinkedIn deep-dives, and open-source project walkthroughs. All practical. All production-grade. devopsshack.com